Parallel Processing on the Web

Arin Boyadjian, Jessica Pinto, Kaitlin Yen

Computer Science Department, California State Polytechnic University, Pomona

boyadjian@cpp.edu
jmpinto@cpp@cppedu
kaitlinyen@cpp.edu

***Abstract*— As the demand for data grows in tandem with the rapid advancement of artificial intelligence (AI) and machine learning (ML) technology, methods of extracting and collecting data are becoming increasingly important. With the multitude of information available on the web, it is not surprising that web scraping has become a popular method of creating datasets to train various AI models and use for analysis. However, such abundance of data is also the reason web scraping can be a tedious task, requiring heavy computation and placing a strain on resources. Hence, our research focuses on the implementation of parallel processing applied to the task of web scraping. Our experiment scrapes data from Wikipedia and Reddit, both websites chosen due to their differing architectures in an attempt to determine whether such differences are reflected in the web scraper's performance. Comparing the performance of two web scrapers using the forking and threading algorithms against baseline sequential scraping shows that threading processes results in the best performance, as measured by scraping time.**

***Index Terms*— Parallel Processing, Web Scraping, HTML, Threading, Forking, CPU.**

I. Introduction

Our world today is becoming increasingly reliant on data in all its myriad forms. With the ability to be applied to countless tasks from social media to artificial intelligence models, it is quickly becoming an integral part of the current technological domain. The importance of data has led to a deeper consideration of data mining tools such as web crawlers and web scrapers–the latter of which we will focus on in this paper.

A. Problem Description

As the amount of information available online through various eclectic sources continues to grow with modern advancements, it is becoming difficult to scrape complicated websites for data that can be applied to a number of different tasks. While attempts to scrape websites today are not unsuccessful, they are not necessarily the most efficient. The appeal of web scrapers comes from the circumvention of traditional data collecting methods such as surveys or manual collection. However, if web scrapers require an equal amount of work for the same result, it can hardly still be considered a better alternative.

When pulling mass information from websites to create datasets, visualize trends, or understand performance, it can often be a time consuming process, with long execution times due to I/O bound operations. Ultimately, it leads to an inefficient use of system resources and presents a challenge for those that rely on web scrapers for large-scale data collection.

B. Proposed Solution

Parallel processing offers a solution to this issue. By having multiple web requests sent and processed simultaneously instead of having to wait for each individual request to complete before starting the next, idle time and overall efficiency can be improved. Multithreading allows multiple threads to share the same memory space while executing tasks concurrently, whereas multiprocessing uses separate processes with independent memory spaces to perform tasks in parallel. This experiment will provide a comparative analysis of the two parallel approaches.

II. Background

A. Parallel Processing

In serial computing, a problem is divided into separate instructions that are executed one after the other on a single processor, which means that each instruction must be completed before moving on to the next. With parallel processing on the other hand, multiple processing units are utilized to solve a problem. The task is broken down into several independent sub-problems and computed at the same time across several processors or threads in parallel, hence the term *Parallel* Processing [5].

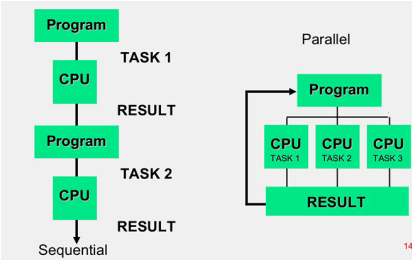

Figure 1.1: Serial versus parallel processing

Parallel processing in 2025 is extensively utilized, and can be found in various applications, including software, business, finance, and entertainment. Some examples of these applications include, but are not limited to:

Unity (Game Engine): Supports multithreading in order to create high quality and high detail games without slowing down the full experience.

Apple: Apple's new M4 chip is equipped with 10-Core GPU, 10-Core CPU and a neural engine that can complete 38 trillion operations per second. This allows for faster download and performance times.

Despite its ubiquity, there are still certain criteria that a program must meet in order to truly optimize a parallel processing enhancement. If a computer's architecture cannot support parallel computing, attempts to implement it through software will prove futile.

B. *Web Scraping*

The term web scraping refers to the "procedure of automatic extraction of data from websites using software" and it can be an extremely useful tool when information is not present in a machine readable format such as a JSON or XML file [6]. Web scrapers are especially useful when pulling information from dynamic websites, the content of which is updated in real-time. On the other hand, static websites maintain versions updated much less frequently. Web scrapers can nevertheless be applied to both types of websites and produce identical results.

Over the years, the evolution of web scrapers from manual operations using Perl-based scripts and text matching to automations that create complete datasets independently has made it a popular tool for a number of tasks. These include data analytics, cloud computing, cybersecurity, and artificial intelligence [6]. The low monetary costs associated with web scraping add to its appeal, especially considering the fact that it is used by numerous organizations and individuals with limited resources at their disposal. Of course, web scraping is often enhanced by the use of Application Programming Interfaces (APIs) that allow for smooth communication between the website and a data fetching program. However, APIs tend to come with their own costs, undermining the goal of affordable data collection that is achieved with web scraping.

Another important issue is the legality of web scraping since certain websites do not allow unmonitored collection of private or copyrighted data. An example of such a website is LinkedIn, which contains an abundance of personally identifiable information (PII) that cannot be scraped. Most web scraper constraints can be found in website service agreements, however it is often difficult to regulate web scraping or mitigate its ethical concerns.

C. *Parallel Processing on the Web*

Since web scraping is primarily an input/output (I/O) bound process, the time it takes for a scraping process to complete is heavily dependent on the amount of time it spends waiting for the I/O operations to complete rather than an inherent lag in the speed of the CPU.

In a serial scraper, another important aspect to consider is the network speed, which could impact the scraper's performance. Even if the utilized computer is fast, a slower network leads to the program spending more time being idle, eventually creating a bottleneck of requests waiting for responses. Parallel processing solves this issue by allowing more than one web request to occur at the same time. Even if the network is slow, at the very least, there is less risk of a bottleneck occurring and thus, the program crashing.

Having multiple requests being handled concurrently allows the scraper to make more efficient use of time that would otherwise be wasted waiting for a response. While conducting the experiment, the goal is to observe a noticeable decrease in total execution time when using parallel processing compared to a serial execution. An increase in throughput is also expected, which means more webpages are scraped per second.

As for state of the art, it has been difficult to find a web scraper that specifically employs parallel processing in a manner similar to our paper. However, in 2024, Weijie Jiang proposed a multi-threaded web crawling model that divides scraped data in several tasks, then assigns each task to a thread [7]. Compared to the single-threaded web crawler, this multi-threaded framework reduces the processing time of the web crawler by 77.364 seconds, which supports our hypothesis that parallel web scraping will increase performance from basic sequential scraping.

III. Methodology

To answer the question of whether parallel processing improves the performance of web scrapers, we set up an experiment that fetches information from two websites: Wikipedia [3] and Reddit [4]. We chose these websites specifically because when it comes to the process of web scraping, there are strict limits on what information can be pulled and where it can be taken from. Wikipedia and Reddit are two websites that are open for web scraping and have less constricting limits compared to other websites we looked into such as LinkedIn and Amazon. Restrictions placed on these websites are largely due to the sensitive information posted to the sites and are implemented in an effort to protect the privacy of users on the site.

The difference in the structural layout of the HTML code of both websites were also significant for analysis. While Wikipedia's source code is easily readable and provides relatively simple access to the information we

were interested in fetching, Reddit's source code is more complex and thus, the site lended itself to information being pulled in the form of JSON files rather than plaintext, as was being done with Wikipedia. Moreover, since the basic code for each website was written in HTML and only the format of information that was fetched varied, this allowed us to analyze whether the type of information being pulled using parallel processing also affects its performance.

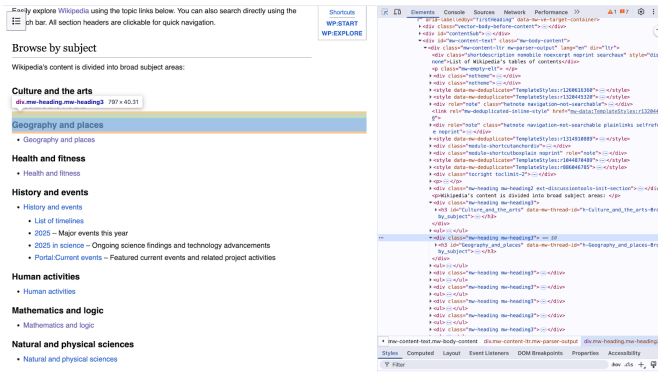

Figure 3.1: Wikipedia Contents page and HTML code

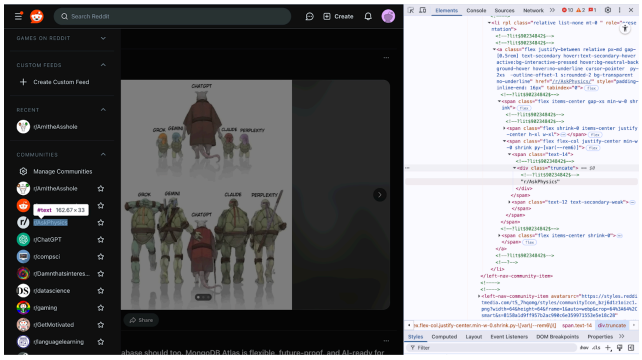

Figure 3.2: Reddit Homepage and HTML code

To fetch information from Wikipedia, we used the `requests_html` library, which is used to send HTTP requests such as GET, POST, and DELETE to web servers and work with APIs and other web scraping tools [2]. Reddit scraping was fetched via a similar library `url_requests` library, which applies the same functions but abstracts the methods more. By using these libraries, we eliminate the potential to deal with common web hurdles such as cookies, sessions, headers and other authentication methods. In our code, we provided the "starting point" URLs which specify the webpage with all the links from which to branch off, exemplified in Figure 3.3.

```
#wiki scraper function
def wiki_scrape_page(title, queue=None):
    url = f"https://en.wikipedia.org/wiki/Wikipedia:Contents/{title}"
    try:
        response = session.get(url)
        threadOverview = response.html.find("h2") #get all the <h2> elements
```

```
def child_fetch_top_posts(subreddit, results, limit=10):
    json_url = f"https://www.reddit.com/r/{subreddit}/top.json?limit={limit}&t=all"
    headers = {'User-Agent': 'Mozilla/5.0 (compatible; Python WebScraper 1.0)'}
    req = urllib.request.Request(json_url, headers=headers)
```

Figure 3.3: Wikipedia start URL (top) and Reddit start URL (bottom)

Using these URLs, we scraped each website for comparable amounts of information. From Wikipedia, we took the subject page's title and the first paragraph from the brief description provided. The resulting display in our graphical user interface (GUI) is shown in Figure 3.4.

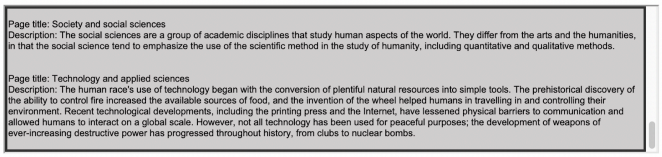

Figure 3.4: Graphical user interface showing data scraped from Wikipedia.

Since there are a total of 13 Subject pages provided by Wikipedia, we used 13 as the maximum number of posts to pull from Reddit–specifically from one Subreddit. When scraping Reddit, we pull the top 13 posts from one Subreddit page, including the post title, post author, upvotes, number of comments, the post URL, and post content in the form of text and emojis. The reason for fetching all these features is to pull a similar amount of information to what is being fetched from Wikipedia for comparable analysis. The resulting display in our GUI for Reddit is shown in Figure 3.5.

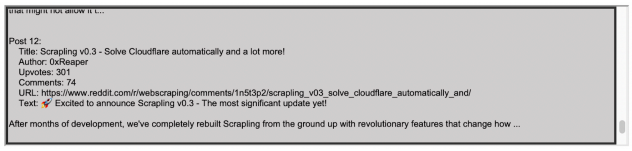

Figure 3.5: Graphical user interface showing data scraped from the subreddit r/webscraping.

The graphical user interface provides users with the ability to choose between a) a scraping method and b) the site to scrape from. Once a user selects their configuration and clicks on "Run Scraper", a model of the processing type is shown along with the total processing time, and the data that has been scraped from the site. The initial GUI along with the results are shown in Figure 3.6.

For testing purposes, we focused on analyzing the amount of time it takes (in seconds) for each website to

scrape a specific amount of information. Our initial testing set consisted of 13 Wikipedia content pages and the top 13 posts on the webscraping subreddit page. Next, we tested the performance of the web scraper by pulling larger amounts of data to examine whether the amount of data correlates with the performance of a certain algorithm. The results of our experiment are expounded in Section IV.

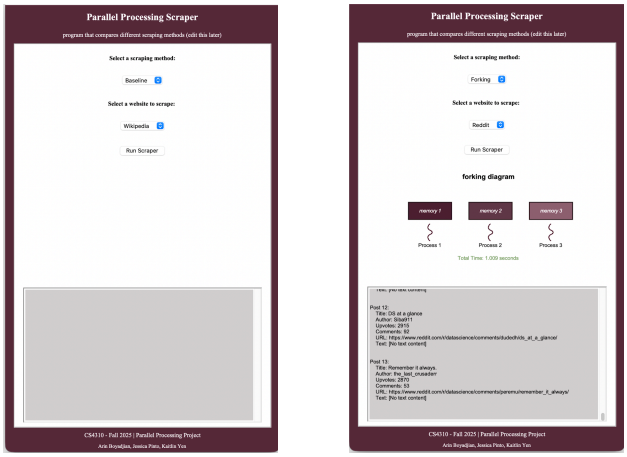

Figure 3.6: Landing page of our application (left) and results after scraping Reddit using the forking algorithm (right).

IV. Results

Our baseline results are shown in Table 4.1 below, which come from running both the Wikipedia and Reddit web scrapers with no modifications. This means that all processing is sequential and no threads or processes are created for parallelization.

	Wikipedia	Reddit	Ranking
Baseline	0.954	1.446	Slowest
Multi Threading	1.228	0.553	Second
Forking	0.597	0.668	Fastest

Table 4.1: Method Sub-Process Comparison. Average execution time by Sub-Process. Number of Sub-Processes (pages scraped): 10. Tests ran: 30

Given these preliminary results, the parallel processing algorithms on average, performed better than the sequential (baseline) processing program. However, the multi-threading program for Wikipedia performed slower than the baseline prompted questions as to why sequential testing may be faster in certain cases. Additionally, the Reddit times across all web-scraping programs were consistently slower than that of the Wikipedia programs.

Upon further investigation, the Wikipedia and Reddit programs were web-scraping different sizes per sub-process. Wikipedia was scraping null text and only taking in a maximum of 120 characters per page for a maximum of 13 pages. Reddit was scraping text and a maximum of 120 characters per post, multiplied by 10 posts, per subreddit given (maximum 10). Therefore, Reddit was scraping considerably more information than that of Wikipedia.

Thus, after modifying both programs to scrape the two websites the parameters to test are as follows:

1. Website Scraped: Wikipedia, Reddit
2. Number of Sub-Processes: Number of Wikipedia Pages, Number of Subreddits
3. Size per Sub-Process: Text length per Wikipedia Page, Number of Posts per Subreddit (120 characters scraped per post)

New tests will evaluate the difference between the number of sub-processes and the sizes per sub-processes. The following are results comparing the number of sub-processes by website scraping.

A. *Sub-Processes Comparison*

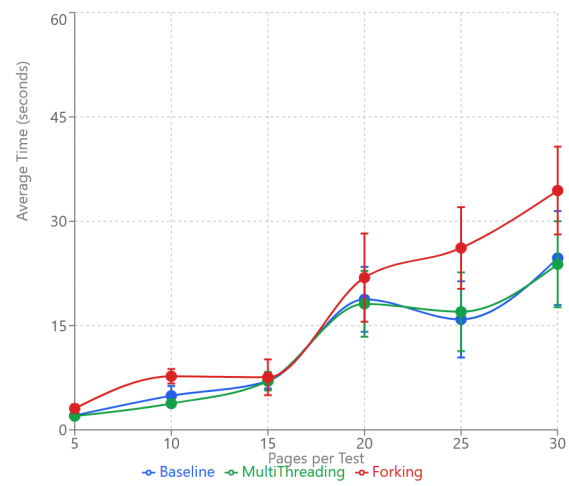

Figure 4.1: Wikipedia Performance Comparison. Average execution time by number of pages (Sub-Processes) given the same text length (Sub-Process size).

According to Figure 4.1, multi-threading performed the best for smaller page counts (5-10 pages), where forking performed about twice as long as baseline and multi-threading. All methods start to show similar performance at 15 pages per test. Variability (error bars) increases with larger page counts. Forking consistently takes longer, especially as page count increases.

Overall trends for total average turnaround time look linear as sub-processes increase regardless of parallelism, with the exception of a small number of sub-processes (5-15). Scraping 100 Wikipedia pages would take about 10 times longer than scraping 10 Wikipedia pages.

B. *Sub-Process Size Comparison*

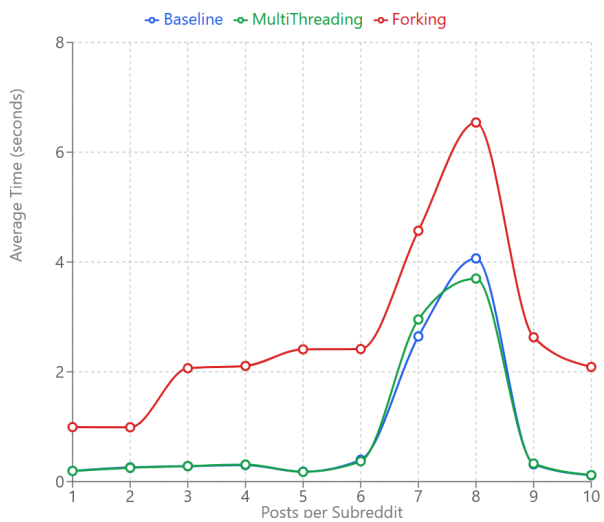

Figure 4.2: Reddit Method Sub-Process Size Comparison. Average execution time by number of posts given the same number of subreddits. Number of Sub-Processes (Subreddits scraped): 1. Tests ran: 30.

According to Figure 4.2, forking consistently performed worse than baseline and multi-threading, which resulted with nearly exact values. Baseline and multi-threading performed the best for up to 6 posts per subreddit. All methods showed a spike in average time for 7-8 posts per subreddit before dropping to similar performance at 9-10 posts like that in the beginning. Variability (error bars) also increases with larger page counts as shown in Figure 4.1.

C. *Discussion*

Results did not match the expectations as forking performed worse compared to the baseline programs for both Wikipedia and Reddit. Multi-threading proved beneficial in both websites for web-scraping as well as per test: increasing the number of sub-processes and the size per sub-process run. Wikipedia times also on average were slower than that of Reddit, even after accounting for test consistency (sub-process size and number of sub-processes ran).

Inaccuracies given Wifi bandwidth and consistency, processes running in the background, and other factors may have affected these results. Further tests to confirm these results on different days and locations are needed in future developments.

Reddit testing also had trouble with fetching posts. As the number of posts scraped increased, all Reddit programs ran into more errors fetching data from the site, which also may have skewed results to faster times as the sub-processes would return nothing. This would explain the difference in times between Wikipedia tests and Reddit tests, as Wikipedia parallel programs checked for "noise" paragraphs whereas Reddit needed a failsafe for inconsistent fetching. Next steps for checking return values and re-running sub-processes are needed for generalisable results.

V. Conclusion

A. *Challenges*

Throughout the development of the project, our team encountered certain technical challenges, primarily related to the integration of the parallel processing techniques with the graphical user interface (GUI). These challenges include:

1. *Multiprocessing causing multiple GUI Windows:* One of the first challenges that were encountered was during the implementation of the forking version of the web scraper. Instead of it running silently in the background and displaying the output in the GUI text box, the program unexpectedly opened multiple GUI windows; one for each of the processors. The number of processors used for testing was 13, thus 13 separate windows. This occurred due to the macOS and Windows system, where Python uses a spawn-based multiprocessing model, which means that each new process re-executes the entire script. Since the GUI code was initially written at the top level of the program, every child process created its own GUI instance. The issue was fixed by moving the full GUI code inside the following conditional block:
    a. `if __name__ == "__main__":`

    This ensures that only the main processes execute the GUI code, which prevents the child processes from creating extra windows. Such a change allowed multiprocessing to function correctly without interfering with the user interface.

2. *Forking results not displaying on the GUI:* After finally resolving the multiple new GUI window issue, the forking results would still not display on the results display box, whereas the baseline and the multithreading versions did. This was due to the fact that multiprocessing does not share memory between processes. Each child process operates in its own independent memory space, which is why Tkinter (Python's built-in GUI framework) could not directly

update its widgets and return the values to the main process the same way it did with threads. To solve this issue, inter-process communication (IPC) was required. A `multiprocessing.Queue` was implemented to allow the child processes to send their scraped results back to the main process. Once this was done, the main process was able to retrieve these results from the queue and safely update the GUI using `root.after()`, which made sure the UI updated safely.

3. *Image caption vs. description parsing:* Another challenge that arose during data extraction, was from the way Wikipedia structures its content. During most of the cases, the scraper was able to detect the correct HTML tags, however, in the cases where an image appeared at the top of the page, the scraper would extract the image caption instead of the actual page description. This resulted in text still being returned, though it was not what the scraper was meant to be doing. The reason this happened was because both image captions and article descriptions are often enclosed with similar HTML tags (example: <p> tags), which makes it difficult to distinguish between the two. A simple way to avoid this issue was to check if the tag had less than 50 words, then it would be considered an image tag, otherwise, it would be considered a description tag and the output would be displayed in the GUI's output section.

B. *What we Learned*

Through this project, the team learned how to work with various web scraping tools, such as the `requests_html` and `json` Python libraries. Initially, our goal was to use a more popular web scraper such as Beautiful Soup or Scrapy, which makes the task of web scraping extremely simple due to their larger arsenal of available tools for parsing HTML. However, for our specific case of scraping information from Wikipedia and Reddit, `requests_html` was still the better option because it is equipped to deal with dynamic websites, where information continues to update after the initial loading of the page [2]. Had we not chosen to scrape dynamic websites like Wikipedia or Reddit, we could have used Scrapey or Beautiful Soup as our web scraper. This is not to say that it is impossible to scrape dynamic websites using these tools, but rather that `requests_HTML` is the optimal library for such a task.

Another lesson learned through this project was how to implement processes and threads for a web based task. Since Python provides a threading and multiprocessing library, the team utilized these functions to create individual processes and threads for each new post or page that was scraped. By defining functions for each scraping method and website, we were able to experiment with the amount of information being pulled and displayed to the user.

Ultimately, our experiment showed that the incorporation of parallel processing certainly improves web scraping performance. While both forking and threading noticeably improve the time efficiency of web scraping on both Wikipedia and Reddit, this enhancement is optimal using multithreading, most likely due to the lightweight nature of threads versus heavier processes.

VI. References

[1] "Python Requests." *GeeksforGeeks*, GeeksforGeeks, 31 July 2025, www.geeksforgeeks.org/python/python-requests-tutorial/.

[2] "HTML: HTML Parsing for Humans (Writing Python 3)!" *ReadTheDocs*, requests.readthedocs.io/projects/requests-html/en/latest/. Accessed 5 Dec. 2025.

[3] "Contents." *Wikipedia*, Wikimedia Foundation, 4 Oct. 2025, en.wikipedia.org/wiki/Wikipedia:Contents.

[4] "Reddit - the Heart of the Internet." *Reddit*, www.reddit.com/. Accessed 6 Dec. 2025.

[5] Gossett, Stephen. "12 Parallel Processing Examples to Know." *Built In*, builtin.com/hardware/parallel-processing-example. Accessed 5 Dec. 2025.

[6] Khder, Moaiad Ahmad. "Web Scraping or Web Crawling: State of Art, Techniques, Approaches and Application." *International Center for Scientific Research and Studies*, 2021, www.i-csrs.org/Volumes/ijasca/2021.3.11.pdf.

[7] Jiang, Weijie. "A Novel Multi-Threaded Web Crawling Model." *arXiv.Org*, 9 May 2024, arxiv.org/abs/2407.10440.